

허니팟 기반 사이버 공격 분석 시스템

공격자를 유인해 수집한 영문 보안 로그를, 규칙 기반 분류와 한국어 LLM 해설을 거쳐 비전문 관리자가 바로 읽고 대응할 수 있는 위협 보고로 바꾸는 통합 분석 파이프라인.

기간	기준일	리비전
2026-04 ~ 2026-09	2026-09-09	75a8189 (main)
기반	상태	
T-Pot 24.04.1 (Telekom Security)	파이프라인 4단 가동	

목차

- 허니팟이란 · 프로젝트 목표
- 배경과 문제 정의
- 진행 경과와 방향 전환
- 시스템 아키텍처
- 기술 스택과 구현
- 핵심 기술 과제 1 — ML 분류기
- 핵심 기술 과제 2 — LLM 판정
- 조용한 실패 네 건
- 검증 결과
- 한계와 개선 방향
- 결론

01 허니팟이란 · 프로젝트 목표

1-1. 허니팟(Honeypot)이란

일부러 취약하게 열어둔 **미끼 시스템**이다. 실제 서비스가 아니므로 여기에 들어오는 접속은 정의상 전부 비정상이고, 그래서 **오탐이 거의 없는 공격 정보**를 얻을 수 있다.

방화벽이나 백신은 "정상 트래픽 속에서 공격을 골라내는" 문제를 푼다. 정상과 공격이 섞여 있으니 오탐과 미탐이 늘 따라온다. 허니팟은 문제를 뒤집는다. **아무도 정상적으로 접속할 이유가 없는 시스템**을 인터넷에 노출해 두면, 들어오는 모든 접속이 곧 공격 시도다.

	일반 보안 장비	허니팟
목적	공격을 막는다	공격을 관찰·기록 한다
트래픽	정상 + 공격 혼재	사실상 전부 공격
오탐	구조적으로 발생	거의 없음
얻는 것	차단 로그	공격 수법·도구·출처·행동 전체

예를 들어 SSH 허니팟 `cowrie` 는 가짜 셸을 제공한다. 공격자가 로그인에 성공했다고 믿고 `wget` 으로 악성코드를 내려받거나 리버스셸을 시도하면, **그 명령어가 그대로 기록된다**. 실제 시스템은 전혀 위험하지 않으면서 공격자의 수법을 통째로 확보하는 것이다.

왜 중요한가

허니팟 데이터는 **실제로 지금 우리 IP 대역을 노리는 공격**의 기록이다. 어느 나라에서, 어떤 계정 사전으로, 어떤 악성코드를 심으려 하는지가 그대로 남는다. 위협 인텔리전스의 1차 자료로 쓰이는 이유다.

1-2. 그런데 기록만으로는 쓸 수 없다

문제는 그 다음이다. 허니팟이 남기는 것은 이런 형태다.

```
{ "eventid": "cowrie.login.failed", "username": "root",
  "password": "123456", "src_ip": "203.0.113.10",
  "dst_port": 22, "protocol": "ssh" }

{ "alert.category": "Attempted Administrator Privilege Gain",
  "alert.signature_id": 2010935, "payload_printable": "..."}

```

이것을 읽고 "무슨 일이 일어났고 무엇을 해야 하는가"로 옮기려면 허니팟별 이벤트 스키마와 MITRE ATT&CK 체계를 이미 알고 있어야 한다. 수집은 자동화됐지만 해석은 여전히 전문가 몫이라는 뜻이다.

1-3. 프로젝트 목표

허니팟이 모은 영문 원시 로그를, 보안 비전문가인 시스템 관리자가 그대로 읽고 바로 조치할 수 있는 한국어 위협 보고로 자동 변환한다.

#	목표	구체적 산출
1	자동 분류	모든 이벤트에 공격 유형 라벨(정찰·무차별대입·악성코드·침입)과 MITRE 위협 점수 0-100 부여
2	한국어 해설	고위험 이벤트에 대해 무슨 일인지 요약 + 지금 할 조치를 한국어 문장으로 생성
3	한눈에 보는 화면	한국어 대시보드, 실시간 공격 지도, 기간별 PDF 리포트
4	기존 자산 보존	T-Pot 본체를 수정하지 않는 사이드카 방식 — 업스트림 업데이트와 충돌 없음

1-4. 이 시스템으로 할 수 있는 것

완성된 파이프라인은 다음 질문들에 한국어로 답한다.

사용자	물어볼 것	시스템이 주는 것
시스템 관리자	어젯밤 우리 서버에 무슨 일이 있었나	24시간 요약 — 총 이벤트 · 공격 분류 · 고위험 건수, 한국어 요약문
시스템 관리자	이 공격, 지금 뭘 해야 하나	<code>solution_ko</code> — "해당 IP를 차단하고 SSH 계정 비밀번호를 변경하십시오" 같은 실행 가능한 조치
보안 담당자	어떤 유형 공격이 늘고 있나	유형별·허니팟별 추이, MITRE 기법 분포, 시간대별 히스토그램
보안 담당자	어디서 들어오나	실시간 공격 지도 — 출발지 국가에서 센서로 향하는 공격 아크
관리 책임자	보고서가 필요하다	기간 선택 → 한국어 PDF 리포트, CSV 내보내기

중소 규모 조직의 위협 조기 경보. 전담 보안 인력이 없는 조직이 자사 네트워크 대역에 허니팟을 하나 띄워 두면, 실제로 자기 대역을 노리는 공격이 무엇인지 한국어로 통보받는다. 대학·연구실 네트워크, 사내 인프라 모니터링, 보안 교육용 실습 환경이 직접적인 적용 대상이다.

02 배경과 문제 정의

1장에서 본 문제를 T-Pot 이라는 구체적인 플랫폼 위에 놓고 다시 정의한다. 무엇이 이미 해결돼 있고 무엇이 비어 있는지를 층위별로 가른다.

2-1. T-Pot 이 주는 것과 주지 않는 것

T-Pot 은 Telekom Security 가 공개한 허니팟 통합 배포판으로, 25종 내외의 허니팟과 Suricata·p0f 같은 패시브 센서를 하나의 Docker 스택으로 묶고 수집 로그를 Elasticsearch 에 적재한다. 인프라로서는 완성도가 높다.

그러나 운영자가 마주하는 화면은 **영문 Kibana 대시보드**이고, 그 안의 항목은 `cowrie.login.failed`, `alert.category: Attempted Administrator Privilege Gain` 같은 원시 이벤트다. 이것을 읽고 "무슨 일이 일어났고 무엇을 해야 하는가"로 옮기려면 허니팟별 이벤트 스키마와 MITRE ATT&CK 체계를 이미 알고 있어야 한다.

2-2. 문제

층위	T-POT 기본	필요한 것
수집	25종 허니팟 → ES 적재 충분	—
분류	없음. 원시 이벤트 그대로	공격 유형 라벨 + 위협 점수
해석	없음	한국어 요약 + 실행 가능한 대응 방안
제시	영문 Kibana	한국어 대시보드, 권한 분리, 리포트

즉 수집은 이미 해결됐고, 분류·해석·제시가 비어 있다. 이 셋을 채우는 것이 본 프로젝트의 범위다.

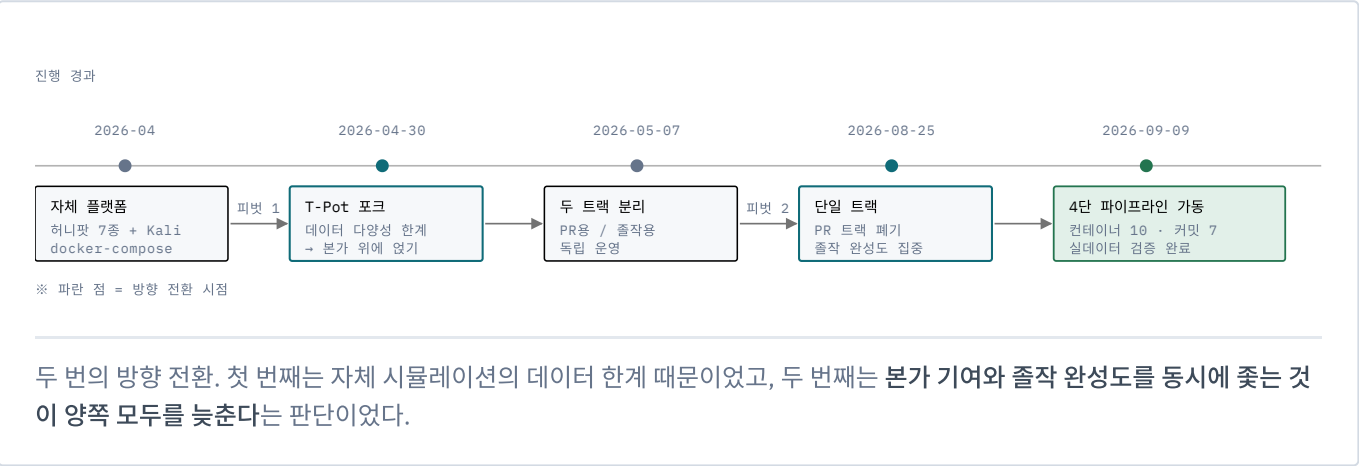
2-3. 설계 원칙

- **사이드카 방식** — T-Pot 본체(허니팟·Logstash·Kibana)를 수정하지 않는다. ES 를 읽고 자기 인덱스에만 쓰는 컨테이너를 덧붙인다. 업스트림 업데이트와 충돌하지 않는다.

- **단계별 인덱스 분리** — logstash-* (원본) → ml-analysis-* (분류) → llm-analysis-* (해설).
각 단계가 독립적으로 재실행·검증 가능하다.
- **실패 시 축소, 중단 아님** — 모델이 없으면 규칙으로, LLM 이 죽으면 결정론적 요약으로 내려간다. 문서가 조용히 사라지지 않는다.

03 진행 경과와 방향 전환

이 프로젝트는 두 번 방향을 크게 틀었다. 두 번 다 측정 결과가 계획을 이겼다.



3-1. 피벗 1 – 자체 플랫폼에서 T-Pot 포크로 (2026-04-30)

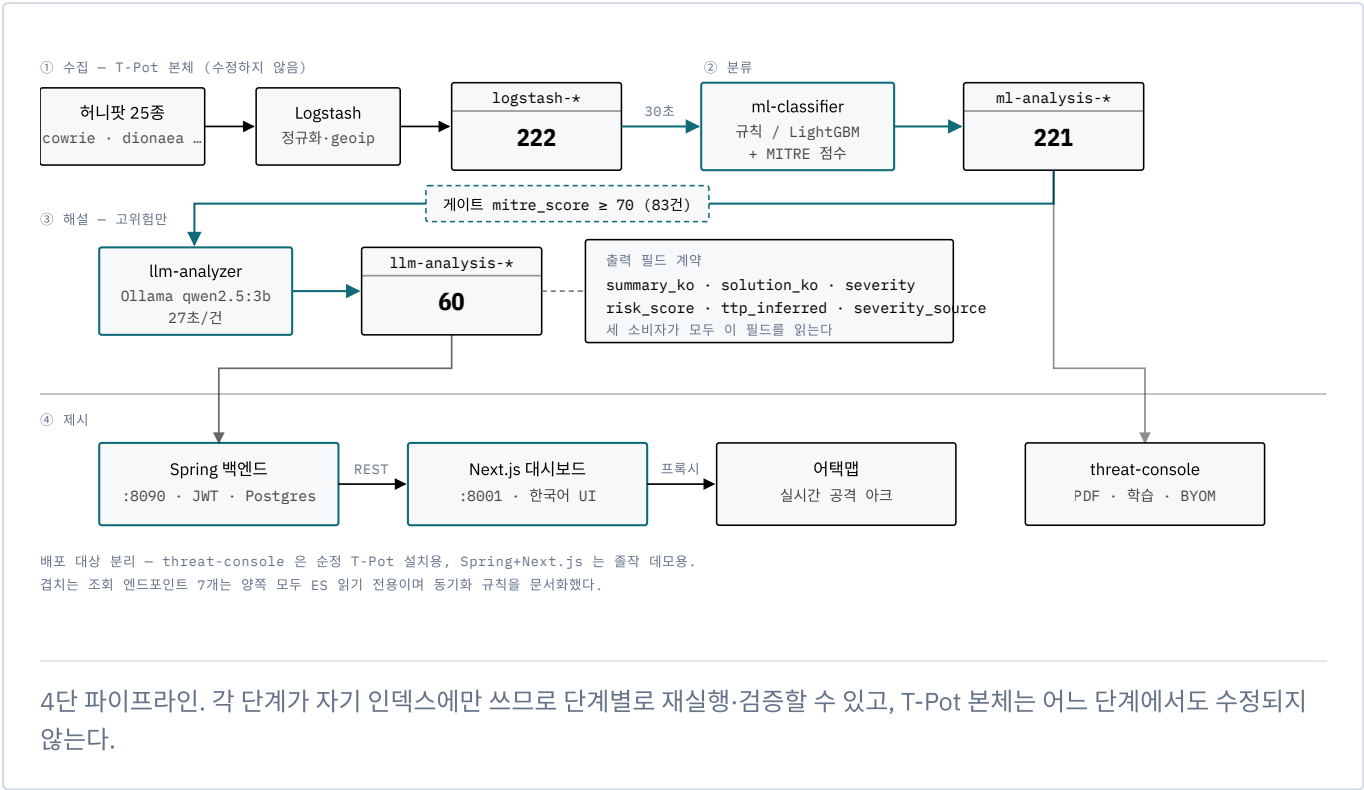
초기 구상은 허니팟 7종과 공격자 컨테이너(Kali)를 직접 묶은 자체 플랫폼이었다. 구현은 됐으나 **공격 데이터가 우리가 스크립트로 만든 것뿐**이라 분류 모델을 학습시킬 다양성이 나오지 않았다. 실제 인터넷 공격을 받는 T-Pot 위에 모듈을 얹는 방식으로 전환했다.

3-2. 피벗 2 – 단일 트랙으로 (2026-08-25)

한동안 본가(telekom-security/tpotce)에 PR 을 제출하는 트랙과 졸업작품 시연 트랙을 나눠 운영했다. PR 트랙은 본가 컨벤션(영어 전용, 머신 의존성 0, Spring/Next.js 금지)을 따라야 해서 **같은 기능을 두 벌로 유지**해야 했다.

본가 기여를 목표에서 제외하고 capstone/ 단일 트랙으로 통합했다. 삭제한 PR 트랙은 2,270 파일이었고, 그중 졸작 트랙에 없는 고유 파일은 2개뿐이었다 (사전 보존 후 삭제).

04 시스템 아키텍처



4-1. 실행 스택

컨테이너	포트	역할
capstone-es	19200	Elasticsearch 8.13.2 – 3개 인덱스 보유
capstone-postgres	5433	사용자·프로젝트·공격로그 영속
capstone-backend	8090	Spring Boot – JWT 인증 + ES 조회 6종
capstone-frontend	8001	Next.js 15 – 한국어 대시보드
capstone-ml-classifier	–	배치 분류 데몬 (30초 주기)
capstone-llm-analyzer	–	배치 해설 데몬 (60초 주기)
capstone-ollama	11434	로컬 LLM 추론
map_web / map_data / map_redis	64299	실시간 공격 지도

`docker compose up -d` 한 번으로 전체가 기동한다. ES 를 스택 안에 두고 `depends_on:`
`service_healthy` 로 기동 순서를 보장해, 이전에 백엔드가 의존성보다 먼저 떠서 반복 재시작하던 문제를

제거했다.

05 구현 상세

5-1. 코드 규모

ML- CLASSIFIER 줄 1,963 Python	LLM- ANALYZER 줄 (신규) 939 Python	THREAT- CONSOLE 줄 678 Python	SPRING 백엔드 1,597 Java · 45 파일
NEXT.JS 프론트 2,616 TS/TSX · 16 파일			

5-2. 기술 스택과 선택 이유

각 도구가 파이프라인의 어느 자리를 맡는지, 왜 그것을 골랐는지 정리한다.

수집 · 저장 계층 — 이미 있는 것을 쓴다

도구	하는 일	왜 이것인가
T-Pot 24.04.1	허니팟 25종을 하나의 Docker 스택으로 묶은 배포판. Telekom Security 가 공개·유지보수한다	허니팟을 하나씩 붙이면 각각 설정·네트워크·로그 형식이 달라 통합에만 프로젝트가 끝난다. T-Pot 은 그 통합을 이미 끝내놓았다 — 우리는 그 위에 얹기만 하면 된다
Elasticsearch 8.13	수집된 로그를 저장하고 검색·집계하는 문서 데이터베이스	로그처럼 스키마가 제각각인 문서를 그대로 넣고 나중에 집계할 수 있다. <code>terms</code> · <code>histogram</code> 집계 하나로 대시보드 차트가 나온다. T-Pot 이 이미 여기에 쌓고 있어 별도 적재가 필요 없다
Logstash	허니팟별로 제각각인 로그를 공통 필드로 정규화하고, IP 를 좌표로 바꿔(geoip) 붙인다	T-Pot 기본 구성. 우리가 손대지 않아도 <code>src_ip</code> · <code>dest_port</code> 같은 공통 필드와 지리 정보가 채워진 채로 들어온다

분석 계층 — 비어 있어서 우리가 만든 부분

도구	하는 일	왜 이것인가
규칙 라벨러 rule_label.py	허니팟 종류·이벤트 ID·명령 패턴을 보고 결정론적으로 공격 유형을 판정한다	ML 의 학습 정답 을 만들면서 동시에 풀백 이 된다. 모델이 없거나 신뢰도가 낮으면 자동으로 여기로 내려온다. 현재 운영 중인 분류기이기도 하다
LightGBM	공격 유형 다중분류 모델. 16개 특징(포트·프로토콜·로그인 시도 횟수·명령 패턴 등)으로 라벨을 예측	표 형태 데이터에서는 딥러닝보다 빠르고 정확한 경우가 많다. 학습이 초 단위라 재실험이 쉽고 모델 파일이 3MB 라 컨테이너에 그대로 넣을 수 있다. 다만 이번 데이터로는 규칙을 못 이겨 비활성 상태다(6장)
MITRE ATT&CK	공격 기법을 T1110 (무차별 대입), T1046 (네트워크 스캔) 같은 표준 ID 로 표기하고 위협 점수 0-100 을 매긴다	전 세계 보안 업계가 쓰는 표준 분류 체계다. 자체 점수를 만들면 근거를 설명할 수 없지만, MITRE 기법 ID 는 외부 문서·타 보안 도구와 그대로 연결된다
Ollama qwen2.5:3b	로컬에서 LLM 을 구동하는 런타임. 고위험 이벤트를 한국어로 요약·해설한다	보안 로그에는 내부 IP 와 계정명이 들어간다. 외부 API 로 보내지 않고 자체 호스팅 하면 데이터가 조직 밖으로 나가지 않는다. 제공사 추상화를 뒤서 필요하면 OpenAI·Anthropic·Gemini 로 바꿀 수 있다

제시 계층 — 사람이 보는 화면

도구	하는 일	왜 이것인가
Spring Boot + PostgreSQL	JWT 인증, 프로젝트 단위 권한, 사용자·공격로그 영속화, ES 조회 API 6종	Elasticsearch 에는 인증·권한 개념이 없다. 누가 어느 프로젝트의 로그를 볼 수 있는가 를 다루려면 관계형 DB 와 애플리케이션 레이어가 필요하다
Next.js 15 React	한국어 대시보드, 로그인·회원가입, 공격 로그 조회, 어택맵 프록시	서버 라우트(/api/*)를 프론트와 같은 프로젝트에 둘 수 있어 ES·Ollama 호출을 브라우저에 노출하지 않고 중계한다
Flask threat-console	순정 T-Pot 설치자용 콘솔 — PDF 리포트, 모델 학습 UI, 모델 업로드	T-Pot 만 쓰는 사용자는 Spring·PostgreSQL 을 원하지 않는다. 의존성 없이 드롭인으로 옅히는 가벼운 콘솔이 따로 필요했다
Docker Compose	10개 컨테이너의 기동 순서·네트워크·볼륨을 한 파일로 정의	<code>docker compose up -d</code> 한 번으로 전체가 재현된다. <code>depends_on: service_healthy</code> 로 의존성 순서를 보장해 백엔드가 DB 보다 먼저 떠서 죽던 문제 를 제거했다

이미 잘 되어 있는 것은 만들지 않는다. 허니팟 운영(T-Pot), 로그 정규화(Logstash), 저장·집계(Elasticsearch)는 검증된 도구에 맡기고, 우리는 **비어 있던 분류·해석·제시 세 층**에만 코드를 썼다. 그 결과 신규 코드는 약 7,800줄이고 T-Pot 본체는 한 줄도 수정하지 않았다.

5-3. ml-classifier – 분류 사이드카

`logstash-*` 를 30초 주기로 폴링해 각 이벤트를 분류하고 `ml-analysis-YYYY.MM.dd` 에 기록한다.

출력 필드	의미
<code>ml_label</code>	Recon / Brute Force / Malware / Intrusion / Etc / Normal
<code>ml_is_attack</code>	라벨 기반 자동 판정 (Etc·Normal 이면 false)
<code>ml_multi_conf</code>	0-100 신뢰도 (규칙 폴백 시 100)
<code>mitre_score</code>	MITRE ATT&CK 위협 점수 0-100
<code>mitre_technique</code>	기법 ID (T1110, T1046 ...)
<code>model_used</code>	<code>rule</code> 또는 <code>ml</code>

특징 추출은 16개 축(시각·포트·프로토콜·허니팟·이벤트유형·로그인시도·명령길이·wget/curl/리버스셸 포함 여부 등)으로 이뤄지며, 학습된 모델이 없으면 `rule_label.py` 의 결정론적 규칙으로 자동 폴백한다. 커서는 `(@timestamp, _seq_no)` 복합 키로 `search_after` 를 쓰므로 같은 밀리초 문서가 누락되거나 무한 재처리되지 않는다.

5-4. llm-analyzer – 해설 사이드카 ● 신규 작성

원래 이 모듈은 별도 브랜치에 존재했으나 **소스가 로컬·백업 어디에도 남아 있지 않았다**. 압축 아카이브 28,246 파일 전수 검색에서도 0건이었다(대조: ml-classifier 93건). 소비자 측 필드 계약에 맞춰 새로 구현했다.

동작은 단순하다. `ml-analysis-*` 중 `mitre_score ≥ 70` 인 문서만 골라 한국어 프롬프트로 LLM 에 보내고, 응답을 `llm-analysis-*` 에 적재한다. 설계에서 신경 쓴 지점은 셋이다.

- **비용 게이트** – LightGBM 추론이 마이크로초라면 LLM 호출은 초 단위다. 점수 게이트로 대상을 좁히고 배치를 작게(기본 20건) 잡는다.
- **중복 과금 차단** – 배치를 분석하기 전에 대상 인덱스에서 기존 `_id` 를 조회해 거른다. `create op-type` 만으로는 중복 쓰기는 막지만 LLM 호출은 이미 낭비된 뒤다.

- **제공자 추상화** — Ollama / OpenAI / Anthropic / Gemini 를 지원하며, 프론트엔드의 `llm-providers.ts` 와 **동일한 환경변수명**을 쓴다. `.env` 하나로 사이드카와 UI 가 같은 백엔드를 본다.

5-5. 제시 계층

Spring 백엔드는 JWT 인증과 프로젝트 단위 권한을 담당하고, ES 조회 6개 엔드포인트를 제공한다. Next.js 프론트는 이를 소비해 한국어 대시보드를 그리며, 어택맵은 `server.js` 의 커스텀 프록시로 `/tpot-map/*` 와 `/websocket` 을 `map_web` 으로 중계한다.

06 핵심 기술 과제 1 – ML 분류기

이 프로젝트에서 가장 많은 시간을 쓴 문제이자, 가장 분명한 결론에 도달한 문제다. 결론은 **"현재 데이터로는 ML 이 규칙을 이길 수 없다"**이고, 그것을 세 번의 실험으로 측정했다.

6-1. 증상

재학습한 LightGBM 모델(holdout 정확도 0.998)을 배치하자 주입 샘플 60건이 **전부 Recon** 으로 분류됐다. 신뢰도는 96~99%. `mitre_score` 가 32에 머물러 고위험이 0건이 되고, 결과적으로 **LLM 해설 단계가 아무 일도 하지 못했다.**

6-2. 1차 진단 – 라벨이 허니팟의 함수

학습 데이터를 열어보니 라벨이 사실상 `source_honeypot` 하나로 결정되고 있었다.

허니팟	RECON	BRUTE FORCE	MALWARE	INTRUSION	합계
cowrie	49,412	582	6	0	50,000
fatt	50,000	0	0	0	50,000
p0f	50,000	0	0	0	50,000
dionaea	0	0	23,725	0	23,725
sentrypeer	0	18,694	0	0	18,694

18개 허니팟 중 16개가 라벨 하나에 100% 고정이었다. `rule_label.py` 가 대부분의 허니팟에 대해 이벤트 내용과 무관하게 상수를 반환하기 때문이다(`if src_hp == "p0f": return "Recon"`). 게다가 Brute Force 클래스 20,571행 중 **18,694행(90.9%)이 sentrypeer** 한 곳에서 나왔다.

핵심

모델은 편향된 것이 아니라 **존재하는 유일한 신호를 정확히 학습**했다. 라벨이 규칙의 출력이므로, 모델이 도달할 수 있는 최선은 규칙의 재현이다. 기존 기록의 "0.998은 과대평가"라는 메모가 정확했다.

6-3. 2차 시도 – 재균형 (실패)

`rebalance.py` 를 만들어 (라벨 × 허니팟) 셀 단위 상한을 걸었다. 지름길은 실제로 차단됐다.

- Brute Force 내 sentrypeer 비중 **90.9% → 51.6%**
- cowrie 내 Recon : Brute Force **85:1 → 1:1**
- 재학습 holdout 정확도 0.9971

그런데 정규 케이스 검증은 **3/6 으로 재균형 전과 동일**했다. 원인이 라벨 불균형이 아니라는 뜻이었다.

6-4. 3차 진단 – 특징 결손과 train/serve skew

특징 값의 분포를 세어보니 **16개 축 중 6개가 학습 내내 상수 0**이었다.

특징	0 인 비율	원인
login_attempts	100.0%	CSV 에 컬럼 자체가 없음
login_success	100.0%	//
has_reverse_shell	100.0%	비0 0건
has_curl	100.0%	비0 1건
has_wget	100.0%	비0 5건
special_char_cnt	100.0%	비0 53건

공격 유형을 실제로 구분하는 바로 그 축들이 비어 있었다. 원인은 CSV 내보내기 구조였다.

COWRIE.CSV 이벤트	행수	DST_PORT	행동 정보
session.connect	41,725	23.0 / 22.0 ✓	없음
session.closed	41,724	—	없음
login.failed	2,987	전부 빈값	있음
command.input	125	전부 빈값	있음

포트·프로토콜은 session.connect 행에만 있고 행동은 login.failed / command.input 행에만 있다. **둘이 다른 행에 흩어져 있어** 어느 쪽도 완전한 특징 벡터를 만들지 못한다. 결과적으로 같은 이벤트가 학습과 추론에서 이렇게 갈렸다.

	DST_PORT	PROTOCOL	LOGIN_ATTEMPTS
학습 행 (cowrie.login.failed)	0	0	0
실제 ES 문서 (같은 이벤트)	22	6 (ssh)	25

모델은 **실서비스 문서처럼 생긴 입력을 한 번도 본 적이 없었다.**

6-5. 4차 시도 – 세션 조인 (개선, 그러나 천장)

session_join.py 로 라이브 Logstash 가 하는 일을 CSV 에서 재현했다. 세션 connect 행에서 포트·프로토콜을 거둬 같은 세션의 모든 이벤트 행에 전파하고, 세션당 login.failed 수를 세어 login_attempts 를 만들고, 89.9%를 차지하던 세션 복키퍼 행을 (속성을 거둔 뒤) 버렸다.

COWRIE	조인 전	조인 후
행 수	50,000	3,217
dst_port 비0	48.3%	100%
login_attempts 비0	0%	93.3%
라벨 분포	Recon 49,412 / BF 582	BF 3,073 / Recon 128

노이즈가 겹치자 cowrie 의 실제 성격(로그인 실패)이 드러났다. 실제 ES 문서 127건으로 규칙 대비 일치율을 재니:

모델	일치율	주요 오류
v2 (조인 전)	35.4%	BruteForce→Recon 33 · Malware→Recon 24
v3 (세션 조인)	61.4%	Intrusion→Malware 25 · Malware→Recon 24
v4 (+payload 필드)	61.4%	동일

천장의 정체

v4 가 v3 를 넘지 못한 이유는 명확하다. payload 필드에서 리버스셸 패턴 384행을 새로 얻었는데, 그중 **322행의 라벨이 Recon** 이었다. Suricata 의 규칙 라벨은 `alert.category` 에서 오고 그것이 payload 내용과 어긋난다. **특징이 생겨도 라벨이 그 특징을 부정하므로** 모델은 "리버스셸 → Recon" 을 학습한다.

라벨이 규칙에서 나오는 한 모델의 천장은 규칙 재현(100%)이고, 현재 61.4%다. 커면 운영 품질이 나빠진다. **그래서 규칙 모드를 유지한다.**

다만 규칙이 구조적으로 못 하는 것 하나는 모델이 해냈다 — 정상 HTTP/HTTPS 트래픽을 `Normal` 로 100% 신뢰도로 분류한다(규칙은 `Recon` 이라 답한다). 허니팟 특성상 정상 트래픽이 드물어 실익은 제한적이지만, ML 이 더할 수 있는 값이 무엇인지는 분명해졌다.

07 핵심 기술 과제 2 – LLM 판정

LLM 이 한국어 요약은 잘 쓰는데 위험도 판정은 전혀 못 한다는 것을 측정으로 확인하고, 그 일을 모델에서 회수했다.

7-1. 증상

초기 구현에서 `mitre_score` 92(멀웨어)인 문서에 모델이 `severity: LOW` , `risk_score: 3.0` 을 반환했다. 파이프라인이 이미 `≥ 70` 만 통과시키므로 이것은 **내부 모순**이다. 보안 대시보드에서 고위험을 LOW 로 표시하는 것은 적극적으로 위험하다.

7-2. 측정 – 프롬프트 문제인가 능력 문제인가

고치기 전에 원인을 갈랐다. 두 가지는 해법이 다르기 때문이다. `eval_severity.py` 로 고위험 문서 10건에 프롬프트 두 변형을 돌렸다.

프롬프트	하한 미달	가드 불필요	한자 혼입	소요
current	10 / 10	0%	1 / 10	592s
fewshot (판정 예시 3개 추가)	10 / 10	0%	0 / 10	341s

판정 예시를 넣어도 10/10 그대로였다. 프롬프트 문제가 아니라 3B 모델의 능력 한계다.

7-3. 설계 변경 – 잘하는 일만 맡긴다

severity 를 아예 묻지 않고 `mitre_score` 에서 결정론적으로 유도하며, 모델은 잘하는 것(한국어 요약·대응 방안·MITRE 기법 추정)만 담당하게 했다.

- 프롬프트가 짧아지고 실패 모드 하나가 사라진다
- `LLM_SEVERITY=model` 로 더 강한 모델에 판정을 되돌려줄 수 있다. 그때도 하한 가드는 유지된다 – **모델은 올릴 수는 있어도 내릴 수 없다**
- 항상 `true` 라 무의미하던 `severity_adjusted` 를 `severity_source (score | llm | raised)` 로 교체해, 누가 판정했는지 인덱스에서 바로 읽힌다

설계 원칙으로

이 경험은 일반화된다. LLM 에게 결정론적으로 계산 가능한 값을 묻지 않는다. 요약·설명·번역처럼 언어 능력이 필요한 일만 맡기고, 점수·등급·분류처럼 규칙으로 환원되는 값은 코드가 계산한다. 그래야 실패해도 조용히 틀리지 않는다.

08 조용한 실패 네 건

이번 개발에서 찾아 고친 결함 네 건은 공통점이 있다. **예외도 경고도 없이** 잘못된 값이나 빈 화면을 냈다는 것이다. 테스트가 아니라 실측이 이것들을 드러냈다.

8-1. `_intval("23.0")` 이 0을 반환

CSV 는 숫자 컬럼을 `"23.0"` 으로 내보내는데 `int("23.0")` 은 예외를 던지고, 예외 처리가 0을 반환했다. CSV 학습 경로의 모든 포트와 카운트가 0이 되는데 라이브 ES 문서(정수)는 정상이라, 같은 이벤트를 두 경로가 다르게 봤다. §5-4 의 train/serve skew 를 만든 직접 원인 중 하나다.

```
# 수정 전 - "23.0" → ValueError → 0
return int(v or 0)
```

```
# 수정 후 — float 를 먼저 거친다
return int(float(v or 0))
```

8-2. payload_printable 미조회

명령 텍스트 추출이 `input` / `command` / `message` 만 읽었는데, Suricata 는 공격 명령을 `payload_printable` 에 담는다. cowrie 가 wget 10행·리버스셸 0행을 낼 때 **Suricata payload 에는 891건·443건**이 있었다. 명령 특징의 유일한 실질적 공급원을 놓치고 있었던 셈이다.

8-3. 인덱스 템플릿의 사각지대

소비자들은 `severity` 를 서브필드 없이 집계한다(`terms(field="severity")`). ES 동적 매핑에 맡기면 `text` 가 되어 이 집계가 `Fielddata is disabled` 로 실패하는데, **threat-console** 과 **Spring** 모두 이 예외를 catch 해 빈 차트로 삼킨다. 기동 시 인덱스 템플릿을 등록해 `keyword` 로 고정하는 것으로 해결했다.

그런데 같은 함정이 한 번 더 나왔다. **템플릿은 인덱스 생성 시점에만 적용된다**. 나중에 `severity_source` 필드를 추가하자, 이미 만들어져 있던 인덱스에 동적 매핑으로 `text` 가 붙어 또 조용히 실패했다. 기동 시 기존 인덱스에도 템플릿 필드를 push 하고, 타입이 충돌하면 숨기지 않고 경고하도록 고쳤다.

8-4. 한국어에 한자 혼입

Qwen 계열 모델이 한국어 문장 중간에 한자를 섞는다(`악의대시`). 프롬프트에 "한국어로만" 규칙을 넣어도 10건 중 1건에서 발생했다. **출력 검사**로 전환해, CJK 표의문자가 발견되면 파싱 오류로 처리하고 기존 재시도 경로가 다시 시도하도록 했다. 재시도가 소진되면 결정론적 폴백 문구 (순수 한국어)로 내려간다. 검증에서 신규 문서 12건 중 한자 혼입 0건.

09 검증 결과

9-1. 파이프라인 실측

LOGSTASH-* 222 수집 이벤트	ML-ANALYSIS-* 221 고 위험 83	LLM-ANALYSIS- * 60 status=ok 60	폴백 0 건
한자 혼입 0 / 12 신규			

검증 항목	결과
규칙 기반 분류 분포	Brute Force 96 · Intrusion 45 · Recon 42 · Malware 38
백엔드 엔드포인트 6종	전부 HTTP 200, 실데이터 응답
프론트 4 페이지	<code>/</code> <code>/login</code> <code>/signup</code> <code>/dashboard</code> 200
어택맵	websocket 101 Switching Protocols, 12프레임 수신
severity 집계	HIGH 32 · CRITICAL 28 (템플릿 수정 후 정상 동작)
export CSV	ml 61행 · llm 11행 (HTTP 200)

9-2. 어택맵 복구

어택맵은 두 가지 이유로 죽어 있었다. 첫째, 컨테이너 3종이 `Created` 상태로 한 번도 기동된 적이 없었고 소속 네트워크도 삭제돼 있었다. 둘째, 더 중요하게, `map_data` 는 `logstash-*` 를 `exists: geoip.ip` 로 필터한 뒤 좌표 필드를 읽는데 **주입 문서에 그 필드가 전혀 없었다**. 컨테이너를 compose 에 편입하고 주입 스크립트에 지오 보강을 추가해, Redis 채널로 아크 메시지가 흐르고 프론트 프록시 경유 websocket 으로 수신되는 것까지 확인했다.

9-3. 재현 절차

```
cd /mnt/d/integration/T-POT_PR/capstone
docker compose up -d          # ES → postgres → backend/ml → llm/frontend
python3 ../integration-tests/inject_sample_docs.py \
    --host http://127.0.0.1:19200 --count 60

# 접속: 프론트 :8001 · 백엔드 :8090 · ES :19200 · 어택맵 /tpot-map/
```

샘플 주입 스크립트가 항상 현재 시각으로 문서를 넣으므로, 이전에 필요하던 "ES 타임스탬프 재조정" 절차가 사라졌다.

10 한계와 개선 방향

10-1. ML 분류기 – 실 데이터 학습

현재 한계

규칙 대비 61.4%. 라벨이 규칙에서 파생되므로 모델의 천장은 규칙 재현이고, CSV 내보내기의 구조적 결손(§5-4)은 세션 조인으로도 완전히 메워지지 않는다.

개선 방향: 허니팟을 실제로 운영해 `logstash-*` 에 데이터를 쌓고 `build_dataset.py --from-es` 로 학습한다. Logstash 가 이미 포트·프로토콜·행동 필드를 한 문서에 합쳐놓으므로 train/serve skew 자체가 발생하지 않는다. 이것이 ML 을 켤 수 있는 유일한 남은 경로다.

더 근본적으로는 **규칙과 독립적인 정답 라벨**이 필요하다. 현재 구조에서는 모델이 규칙을 모방하는 것 이상을 할 수 없다. 공개 침해 데이터셋과의 결합이나 수동 라벨링 표본이 대안이 될 수 있다.

10-2. LLM – 모델 상황

3B 모델은 한국어 산문은 쓸 만하지만 위험도 판정은 전혀 못 한다(§6-2). 호스트 RAM 7.8GB 제약으로 8B 모델은 OOM 이 났다.

개선 방향: 메모리 확보 후 8B 재시도 또는 외부 API (`LLM_PROVIDER=anthropic` 등) 전환. 전환 후 `LLM_SEVERITY=model` 로 되돌리면 `severity_source` 분포가 곧바로 효과 측정치가 된다 (`11m` 비율이 높을수록 모델이 제 몫을 한다는 뜻).

처리 속도도 과제다. CPU 추론이 건당 27초라 고위험 48건에 약 5분이 걸린다. 실제 T-Pot 트래픽(시간당 수 천 건)은 따라갈 수 없다. GPU 추론, 게이트 상향, 또는 동시 처리가 필요하다.

10-3. 규칙 라벨러 자체의 개선

§5-2 에서 드러났듯 `rule_label.py` 는 대부분의 허니팟에 대해 내용을 보지 않고 상수를 반환한다. 이는 ML 의 천장을 만드는 원인이기도 하다. cowrie 처럼 이벤트 내용을 실제로 분기하는 로직을 다른 허니팟으로 확대하면 규칙 자체의 품질도, 그 규칙에서 파생되는 라벨의 품질도 함께 올라간다.

10-4. 운영 측면

- **실 T-Pot 연동** – 현재 검증은 합성 샘플 기반이다. `.env` 의 `ES_HOST` 만 바꾸면 실 T-Pot ES 로 전환되도록 설계돼 있으나 실제 연동 검증은 남아 있다.
- **리소스** – 단일 호스트 7.8GB 로 ES·Postgres·Ollama·10개 컨테이너를 모두 돌리기에는 빠듯하다. 시연 전 무관 컨테이너 정리가 필요하다.
- **인증·권한** – Spring 의 프로젝트 단위 권한 모델은 구현돼 있으나 대시보드와의 연결은 부분적이다.

11 결론

영문 허니팟 로그를 한국어 위협 보고로 바꾸는 4단 파이프라인을 구축하고 실패데이터로 동작을 확인했다. 다만 그 과정에서 얻은 가장 큰 성과는 파이프라인 자체가 아니라, **무엇이 되고 무엇이 안 되는지를 측정으로 가른 것이다.**

11-1. 달성한 것

- T-Pot 본체를 수정하지 않는 **사이드카 4종**과 인덱스 3단 분리 구조
- 유실됐던 **llm-analyzer 모듈 재작성**(939줄) — 이전까지 해설 인덱스에 쓰는 주체가 아예 없었다
- `docker compose up -d` 한 번으로 재현되는 **자립형 데모 스택**
- 규칙 기반 분류 + 한국어 LLM 해설 + 대시보드 + 실시간 공격 지도의 **end-to-end 동작**
- 조용히 실패하던 **결함 4건** 발견·수정

11-2. 측정으로 가른 것

질문	답	근거
ML 분류기를 켤 수 있는가	• 아니오	규칙 대비 61.4%. 라벨이 규칙 파생이라 천장이 규칙 재현
재균형으로 해결되는가	• 아니오	편중 해소해도 정규 케이스 3/6 동일 — 원인은 특징 결손
세션 조인이 효과가 있는가	• 부분	35.4% → 61.4% (+26pp), 그러나 약라벨 모순으로 정체
LLM 이 위험도를 판정할 수 있는가	• 아니오	3B 모델 10/10 실패, few-shot 도 10/10 — 능력 한계
LLM 이 한국어 해설을 쓸 수 있는가	• 예	60건 status=ok, 폴백 0, 한자 혼입 0/12
콘솔 둘을 통합해야 하는가	• 아니오	배포 대상이 다름. 겹치는 7개는 읽기 전용이며 드리프트 없음

11-3. 배운 것

세 가지가 반복해서 나타났다.

- **높은 정확도가 동작을 뜻하지 않는다.** holdout 0.998 인 모델이 실제 문서 6건 중 3건을 틀렸다. 학습 분포 안의 수치와 실서비스 입력에 대한 성능은 다른 이야기다.

- **조용한 실패가 가장 비싸다.** 네 건 모두 예외 없이 잘못된 값을 냈고, 그중 둘은 예외를 *잡아서* 빈 화면으로 바꾸는 코드 때문에 더 오래 숨었다.
- **고치기 전에 재는 것이 빠르다.** LLM severity 는 프롬프트를 고칠 뻔했으나, 10건을 재보니 프롬프트 문제가 아니었다. 측정이 아니었다면 효과 없는 수정에 시간을 썼을 것이다.